

AGILE SOFTWARE ENGINEERING (CS303)

TDD & Refactoring Best Practices

Learning Objectives

Understand Test Driven Development (TDD), Refactoring, Extreme Programming (XP), unit testing, code quality improvement techniques, and best practices.

Introduction to Extreme Programming (XP)

Extreme Programming (XP) is an Agile software development methodology that focuses on high-quality software, continuous feedback, teamwork, and frequent releases.

What is Test Driven Development (TDD)?

TDD is a software development practice where tests are written before the actual code. Developers first create a failing test, then write code to pass the test, and finally improve the code structure.

TDD Cycle

1. Write a Test (Red)
2. Write Code to Pass the Test (Green)
3. Refactor the Code (Refactor)

Red-Green-Refactor Model

The Red-Green-Refactor cycle is the foundation of TDD. It ensures code correctness while continuously improving design and maintainability.

Unit Testing

Unit testing verifies individual components or functions of a program. Automated unit tests help detect defects early and improve software quality.

Benefits of TDD

- Improved Code Quality
- Early Bug Detection
- Better Design
- Easier Maintenance
- Increased Developer Confidence

What is Refactoring?

Refactoring is the process of improving internal code structure without changing its external behavior. The goal is to make code cleaner, simpler, and easier to maintain.

Objectives of Refactoring

- Improve readability
- Reduce complexity
- Improve maintainability
- Eliminate duplication

- Enhance performance

Common Code Smells

1. Duplicate Code
2. Long Methods
3. Large Classes
4. Excessive Parameters
5. Dead Code

Refactoring Techniques

- Extract Method
- Rename Variables
- Remove Duplicate Code
- Simplify Conditional Statements
- Replace Temporary Variables

TDD vs Traditional Testing

TDD: Tests are written before code.

Traditional Testing: Tests are written after development.

TDD encourages better design and continuous validation.

Best Practices

- Write small tests.
- Keep tests independent.
- Refactor regularly.
- Maintain test coverage.
- Use meaningful test names.

Advantages of Refactoring

Refactoring improves software quality, maintainability, scalability, readability, and overall system performance.

Real World Applications

Modern software companies use TDD and Refactoring extensively in Agile projects to maintain reliable and scalable applications.

Exam Important Questions

1. Define TDD.
2. Explain the Red-Green-Refactor cycle.
3. What is Refactoring?
4. Discuss Code Smells.
5. Compare TDD and Traditional Testing.

Summary

TDD and Refactoring are key Agile practices that help teams develop high-quality software. TDD ensures correctness through testing, while Refactoring improves code structure and maintainability.